

Code Layout, Readability and Reusability

Starting Date	08 October 2026
Team ID	UNM1611PUNITHA
Project Name	AI STUDY BUDDY API

Instructions

Describe how the project source is organized and evaluate readability/reusability using only structures visible in the supplied code.

Project Layout

Path	Responsibility
index.js	Application bootstrap, middleware, route mounting, health endpoint, error handler and server start.
src/routes/	Separates API endpoint definitions by auth, materials and admin domains.
src/controllers/	Contains business/application logic for authentication, material/AI operations and administration.
src/middleware/	Reusable request-processing logic for authentication/authorization and file uploads.
src/models/	Mongoose schemas for User and Material persistence.
src/utils/	Reusable database, JWT/token and Gemini helper functions.
uploads/	Temporary file-storage directory created when the application starts.

Readability and Reusability Assessment

Area	Assessment
Separation of concerns	Good: routing, controller logic, middleware, models and utilities are separated.
Naming	Generally descriptive names such as protect, adminOnly, generateStudyPlan, uploadMaterial and connectDB make responsibilities clear.
Reuse	askGemini and generateTokens centralize repeated integrations; protect/adminOnly are reusable middleware.
Database modeling	User and Material schemas centralize persistence structure and validation.
Configuration	Secrets and environment-specific values are loaded through dotenv rather than hard-coded in source.
Error handling	A global Express error handler exists, while controllers also return explicit 4xx responses for common cases.
Maintainability concern	Some controller functions are long and combine database access, prompt creation, parsing and persistence; these could be split into smaller services/helpers.

Reusability Checklist

- Reusable Gemini integration is isolated in `src/utls/gemini.js`.
- JWT creation is centralized in `src/utls/tokens.js`.
- Authentication and role authorization are reusable middleware functions.
- Multer upload configuration is isolated from controller logic.
- Mongoose models provide a shared data contract for controllers.
- Route modules keep endpoint registration separate from implementation.
- Environment variables support different development/production configurations.
- Further reuse opportunity: move repeated `Material.findById` + not-found checks into a helper/service.
- Further reuse opportunity: centralize AI JSON parsing and validation for flashcards and quizzes.

Additional Notes / Comments

Documentation accuracy: the project README contains a few statements that differ from the supplied implementation. This filled document prioritizes the actual source code.

Security/readability observation: passwords are hashed, JWT secrets are environment-based, and protected resources enforce ownership. However, the current logout flow does not revoke tokens, and AI JSON responses are parsed directly with `JSON.parse` without a validation layer.

Current scope: the codebase is compact and modular enough for a project-development submission, while the next maintainability step would be extracting AI services, validation, and common resource-access helpers.